



CSE 3131

Compiler Design

Md. Emdadul Haque

Lecturer

Department of Computer Science & Engineering
North Bengal International University



Code generation

Code generation can be considered as the final phase of compilation. The code generated by the compiler is an object code of some lower-level programming language, for example, assembly language. We have seen that the source code written in a higher-level language is transformed into a lower-level language that results in a lower-level object code.

Code generator is used to produce the target code for three-address statements. It uses registers to store the operands of the three address statement.

Example:

Consider the three address statement $x := y + z$. It can have the following sequence of codes:

MOV x, R0

ADD y, R0

Define code generation with suitable example.

Register and Address Descriptors

- A register descriptor contains the track of what is currently in each register. The register descriptors show that all the registers are initially empty.
- An address descriptor is used to store the location where current value of the name can be found at run time.

Write short notes on register and address descriptors.

A simple code-generation algorithm

The algorithm takes a sequence of three-address statements as input. For each three address statement of the form $a := b \text{ op } c$ perform the various actions. These are as follow

1. Invoke a function **getreg** to find out the location L where the result of computation $b \text{ op } c$ should be stored.
2. Consult the address description for y to determine y' . If the value of y currently in memory and register both then prefer the register y' . If the value of y is not already in L then generate the instruction **MOV y' , L** to place a copy of y in L

Write down the code generation algo..

A simple code-generation algorithm (contd.)

3. Generate the instruction **OP z' , L** where z' is used to show the current location of z. if z is in both then prefer a register to a memory location. Update the address descriptor of x to indicate that x is in location L. If x is in L then update its descriptor and remove x from all other descriptor.

4. If the current value of y or z have no next uses or not live on exit from the block or in register then alter the register descriptor to indicate that after execution of $x := y \text{ op } z$ those register will no longer contain y or z.

Write down the code generation algo..

Generating Code for Assignment Statements

The assignment statement
 $d := (a-b) + (a-c) + (a-c)$
can be translated into the
following sequence
of three address code:

$t := a - b$

$u := a - c$

$v := t + u$

$d := v + u$

Code sequence for the
example is as follows:

Statement	Code Generated	Register descriptor Register empty	Address descriptor
$t := a - b$	MOV a, R0 SUB b, R0	R0 contains t	t in R0
$u := a - c$	MOV a, R1 SUB c, R1	R0 contains t R1 contains u	t in R0 u in R1
$v := t + u$	ADD R1, R0	R0 contains v R1 contains u	u in R1 v in R1
$d := v + u$	ADD R1, R0 MOV R0, d	R0 contains d	d in R0 d in R0 and memory

Generate a code for the given expression.

Design Issues

In the code generation phase, various issues can arise:

- Input to the code generator
- Target program
- Memory management
- Instruction selection
- Register allocation
- Evaluation order

Describe the code generation issues.

1. Input to the code generator

- The input to the code generator contains the intermediate representation of the source program and the information of the symbol table. The source program is produced by the front end.
- Intermediate representation has the several choices:
 - a) Postfix notation
 - b) Syntax tree
 - c) Three address code
- We assume front end produces low-level intermediate representation i.e. values of names in it can directly manipulated by the machine instructions.
- The code generation phase needs complete error-free intermediate code as an input requires.

Describe the code generation issues.

2. Target program

The target program is the output of the code generator. The output can be:

- a) **Assembly language** : Assembly language as output makes the code generation easier.
- b) **Relocatable machine language**: It makes the process of code generation easier.
- c) **Absolute machine language**: Absolute machine language as output has advantages that it can be placed in a fixed memory location and can be immediately executed.

Describe the code generation issues.

3. Memory management

- During code generation process the symbol table entries have to be mapped to actual p addresses and levels have to be mapped to instruction address.
- Mapping name in the source program to address of data is co-operating done by the front end and code generator.
- Local variables are stack allocation in the activation record while global variables are in static area.

In static allocation, the position of an activation record is fixed in memory at compile time.

In the stack allocation, for each execution of a procedure a new activation record is pushed onto the stack. When the activation ends then the record is popped.

Describe the code generation issues.

4. Instruction selection

- Nature of instruction set of the target machine should be complete and uniform.
- When you consider the efficiency of target machine then the instruction speed and machine idioms are important factors.
- The quality of the generated code can be determined by its speed and size.

Describe the code generation issues.

5. Register allocation

Register can be accessed faster than memory. The instructions involving operands in register are shorter and faster than those involving in memory operand.

The following sub problems arise when we use registers:

- **Register allocation:** In register allocation, we select the set of variables that will reside in register.
- **Register assignment:** In Register assignment, we pick the register that contains variable.

Certain machine requires even-odd pairs of registers for some operands and result.

- For example, Consider the following **division instruction** of the form:

D x, y

Where,

x is the dividend even register in even/odd register pair, y is the divisor

- Even register is used to hold the remainder.
- Odd register is used to hold the quotient.

Describe the code generation issues.

6. Evaluation order

The efficiency of the target code can be affected by the order in which the computations are performed. Some computation orders need fewer registers to hold results of intermediate than others.

Describe the code generation issues.

Thanks